



Hospital Management System

v2.0 — from working prototype to production-ready

Prepared for jhraihaan — this is your build plan

Date 4 September 2026

Reviewed at commit `12b2062`, branch `main`

Stack Django 6.1 · DRF 3.18 · SimpleJWT 5.5 · React 19 · Vite 8 · Tailwind 4

Effort ≈ 8–11 weeks solo, in 7 phases

1 What this is

You asked me to review the project, find what's wrong, and say what would make it industry standard. I read every backend file and the frontend core and verified each finding against the code rather than assuming it. This document is the result: what's broken, what to build, and the order to do it in.

Start with the good news. Your domain model is sensible — the entities and their relationships are the right ones. The code is clean and readable. You separated `models`, `serializers`, `permissions` and `views` the way DRF intends. The React app is tidy and consistently structured. As a demonstration that you can build CRUD over DRF, it works.

The gap is that a hospital system is not a CRUD app. It holds the most sensitive category of personal data there is, and the current code has no meaningful defence around it. I found **40 issues: 6 critical, 10 high, 16 medium, 8 low**.

The three that matter most

1. Anyone can make themselves an admin. Your public registration endpoint accepts a `role` field from the client. Post `{"role": "admin"}` to it and you are an administrator. Your React registration form has a role dropdown, so this is offered to users directly.

2. Every authenticated user can read every record. Your permissions check roles on writes but allow all reads. Any patient who signs up can list every other patient's address, phone, blood group, diagnoses, prescriptions and bills.

3. Your filters do nothing. `DjangoFilterBackend` is imported but never connected. Every filter the UI sends is silently discarded and the full table comes back, so the screen looks filtered when it isn't.

None of these are hard to fix. They are all a few lines each. But until they are fixed, this application must not be pointed at real patient data.

2 Critical — fix these first

Six issues. Roughly two to three days of work. Nothing else should start before these are done.

CRITICAL C-1 · Anyone can register as an admin

hms/backend/serializers.py:11 · hms/backend/views.py RegisterView

`role` is writable on `UserSerializer` and `RegisterView` is `AllowAny`, so the role a user claims at signup is the role they get. This is complete privilege escalation available to anyone who can reach the site.

Fix. Drop `role` from the registration serializer and force it server-side. Create staff through an admin-only endpoint or the Django admin instead.

```
# serializers.py — registration must never accept a role
class RegisterSerializer(serializers.ModelSerializer):
    class Meta:
        model = User
        fields = ['id', 'username', 'email', 'password', 'first_name', 'last_name']
        # ^ no 'role' — self-signup is always a patient

    def create(self, validated_data):
        return User.objects.create_user(**validated_data, role=User.Role.PATIENT)
```

Done when. Posting `{"role": "admin"}` to `/register/` produces a user whose role is `patient`, and a test asserts it. Role dropdown removed from the React form.

CRITICAL C-2 · Every user can read every record

hms/backend/permissions.py · all viewsets in views.py

`RoleWritePermission.has_permission` returns `True` for anything in `SAFE_METHODS`, and no viewset overrides `get_queryset()`. So role checks apply to writes only — reads are wide open to any logged-in user. A patient who just signed up can `GET /patients/` and download your entire medical registry.

Fix. Scope rows to the requester in `get_queryset()` on every viewset, and stop treating reads as automatically safe. Put the scoping in a shared mixin so a new endpoint inherits it instead of relying on you remembering.

```
# views.py - a patient sees only themselves, a doctor only their own patients
def get_queryset(self):
    qs, user = Patient.objects.all(), self.request.user
    if user.role == User.Role.PATIENT:
        return qs.filter(user=user)
    if user.role == User.Role.DOCTOR:
        return qs.filter(appointments__doctor__user=user).distinct()
    return qs # admin / receptionist see all
```

Done when. A patient requesting another patient's detail URL gets **404**, not 403 — you don't confirm the record exists. Covered by the permission-matrix tests in §6.

CRITICAL C-3 · Patients can edit other people's appointments

hms/backend/permissions.py — AppointmentWrites

`AppointmentWrites` allows write access to any patient or doctor for *any* appointment row. A patient can approve their own booking, or cancel and reschedule a stranger's.

Fix. Add `has_object_permission()` that compares the row's patient or doctor to `request.user`. Approval becomes a separate action available only to staff, not something a patient can set on themselves.

CRITICAL C-4 · Secrets and debug settings committed to git

hms/hms/settings.py lines 23, 26, 28, 130

A hard-coded `django-insecure-` secret key, `DEBUG = True`, empty `ALLOWED_HOSTS`, and `CORS_ALLOW_ALL_ORIGINS = True`. A published secret key lets anyone forge sessions and tokens; `DEBUG` hands stack traces, settings and SQL to any visitor who triggers an error.

Fix. Move all four to environment variables with `django-environ`, commit a `.env.example`, split settings into base/dev/prod, rotate the leaked key, and replace the CORS wildcard with an explicit origin list. Add HSTS, `SECURE_SSL_REDIRECT` and secure cookies in production.

Done when. `manage.py check --deploy` reports zero issues, checked in CI.

CRITICAL C-5 · The admin saves passwords in plain text

hms/backend/admin.py:7

Your `UserAdmin` subclasses `admin.ModelAdmin` instead of `django.contrib.auth.admin.UserAdmin`. That loses the password-hashing widget, so a password typed into the admin form is written to the database verbatim.

Fix. Subclass the real `UserAdmin` and add `role` to its `fieldsets` and `add_fieldsets`. Also register `Bill`, which you never registered at all.

CRITICAL C-6 · Nobody else can install your backend

project root

There is no `requirements.txt`, no `pyproject.toml` and no lockfile anywhere in the repository. The backend is installable only on your machine, and cannot be built by CI, a container, or a reviewer.

Fix. Add `pyproject.toml` with pinned versions and a lockfile, split into runtime and dev dependencies. This is thirty minutes and it unblocks everything in §7.

3 High priority

Real defects and material gaps. Two of these — H-1 and H-2 — are actively breaking things right now.

Issue		What's wrong, and the fix
H-1	Stray tkinter import <code>models.py:1</code>	<code>from tkinter.tix import STATUS</code> sits at the top of your models file — an IDE autocomplete accident. <code>tkinter.tix</code> was removed in Python 3.13 , and <code>tkinter</code> isn't in slim container images, so this breaks both a modern interpreter and any Docker deploy. It's unused. Delete the line.
H-2	Filtering is dead code <code>views.py</code> , <code>settings.py</code>	<code>DjangoFilterBackend</code> is imported but never attached to anything — <code>filter_backends</code> , <code>filterset_fields</code> and <code>DEFAULT_FILTER_BACKENDS</code> appear nowhere in the project. Your Appointments page sends doctor, patient and date params that DRF discards. Register the backends globally and give each viewset a <code>filterset_class</code> . Return 400 on an unknown filter rather than ignoring it.
H-3	No pagination <code>settings.py</code>	Every list endpoint serialises the whole table. At real volume this is both slow and a denial-of-service vector. Set <code>PageNumberPagination</code> with <code>PAGE_SIZE = 25</code> , capped at 100.
H-4	Routes aren't protected <code>App.jsx</code>	The <code><Layout /></code> branch has no auth guard, so a logged-out visitor can open <code>/patients</code> and render the shell. Only the API 401 stops data loading, which gives a broken screen instead of a redirect. Add a <code><ProtectedRoute></code> wrapper that redirects to login and remembers where they were going.
H-5	Sessions die after 5 minutes <code>api.js</code>	You have a request interceptor but no response interceptor. SimpleJWT's default access token lasts 5 minutes and your stored <code>refresh_token</code> is never used, so every session silently breaks. There's also no logout endpoint, so tokens stay valid after signout. Add a 401 interceptor that refreshes once and replays the request — share one in-flight promise so concurrent 401s don't cause a refresh storm.
H-6	Doctors can be double-booked <code>models.py</code>	Nothing stops two appointments at the same time with the same doctor, or a booking in the past, or one against a doctor marked unavailable. No constraint, no <code>clean()</code> . Add a unique constraint on (doctor, datetime) for non-cancelled rows plus validation, and reserve slots with <code>select_for_update()</code> .

H-7	Any doctor can prescribe for any patient <code>serializers.py</code>	Prescriptions are never checked against the appointment's own doctor, so one doctor can write into another's consultation. Validate that <code>appointment.doctor == request.user.doctor</code> and store <code>prescribed_by</code> .
H-8	Middleware duplicated and misordered <code>settings.py:51,57,58</code>	<code>CommonMiddleware</code> is listed twice and <code>CorsMiddleware</code> sits below it. <code>django-cors-headers</code> requires placement <i>above</i> <code>CommonMiddleware</code> or CORS headers go missing on redirects. Remove the duplicate, move CORS up.
H-9	Zero tests <code>tests.py</code>	The file contains only the scaffold comment. None of the authorisation rules above can be proven or protected from regression. This is what makes the security work trustworthy — see §6.
H-10	Registration creates orphaned users registration flow	Signing up as a doctor or patient creates a <code>User</code> but no <code>Doctor</code> / <code>Patient</code> profile row, so the account can't be booked, prescribed to or billed. It exists but is invisible to every feature. Create both inside one <code>transaction.atomic()</code> and backfill existing orphans with a data migration.

4 Medium and low

Not urgent, but this is the difference between "student project" and "professional codebase".

Model hygiene

- No `__str__()` on any model — your admin shows `Doctor object (3)` in every dropdown.
- No `Meta.ordering` — combined with pagination this gives unstable page contents and warnings.
- No `related_name` on any foreign key, so reverse lookups fall back to `_set`.
- `gender` and `blood_group` are free text with no `choices` — you'll get "M", "male", "Male", "b+" and never be able to query it.
- `User.email` is neither unique nor required; `role` has no default, so a user made outside the serializer gets an empty role matching no rule.
- Missing `db_index` on every column you filter by.
- Records are hard-deleted and cascade. **Clinical records must be retained** — deletion has to be an archive flag, not a `DELETE`.

API and performance

- **N+1 queries everywhere.** Your serializers nest `user` and `get_prescription_medicines()` runs a fresh query per row, with no `select_related` or `prefetch_related` anywhere.
- `PrescriptionSerializer` overrides `create()` but not `update()`, so a PATCH carrying medicines silently drops them.
- No API documentation — the contract exists only in the source.
- No rate limiting, so `/login/` is open to unlimited credential stuffing.
- No structured logging, error tracking or health-check endpoint.
- **No audit trail.** Nothing records who read or changed a patient record. This is mandatory in healthcare and the hardest thing on this list to add later — which is why it's in phase 1, not phase 6.

Frontend and repo

- API base URL hard-coded to `127.0.0.1:8000`, so the frontend can't be built for any other environment.
- Tokens in `localStorage` are readable by any injected script.
- Every page hand-rolls `useEffect` + `useState` fetching — no caching, dedup, retry or invalidation.
- The sidebar shows all six modules to every role regardless of permission.
- In `Layout.jsx` the logout button uses `absolute bottom-6` inside a non-relative parent, so it anchors to the viewport rather than the sidebar.

- No error boundary and no 404 page — unknown routes silently redirect to `/appointments` .
- No `.env.example` , licence or contributing guide. Your README advertises PostgreSQL but only SQLite is configured.
- ESLint is configured but never enforced; nothing lints the Python at all.
- The Django app is called `backend` , which blocks splitting it into real bounded apps.

5 What to build

Fixing defects gets you to correct. This section is what gets you to industry standard.

5.1 Roles and the permission matrix

Add a **pharmacist** role — you have a medicines module with nobody who owns it. This table is the specification to implement in `get_queryset()` and test against.

R read all · **R•** read own only · **W** write all · **W•** write own only · **—** no access

Resource	Admin	Doctor	Reception	Pharmacist	Patient
Users & roles	R W	—	—	—	—
Departments	R W	R	R	R	R
Doctors	R W	R / W•	R	R	R profile
Patients	R W	R• own appts	R W	—	R• W•
Appointments	R W	R• W•	R W	—	R• W•
Prescriptions	R	R• W•	—	R• dispensable	R•
Medicines & stock	R W	R	R	R W	—
Bills & payments	R W	—	R W	—	R•
Audit log	R	—	—	—	—

5.2 Data model changes

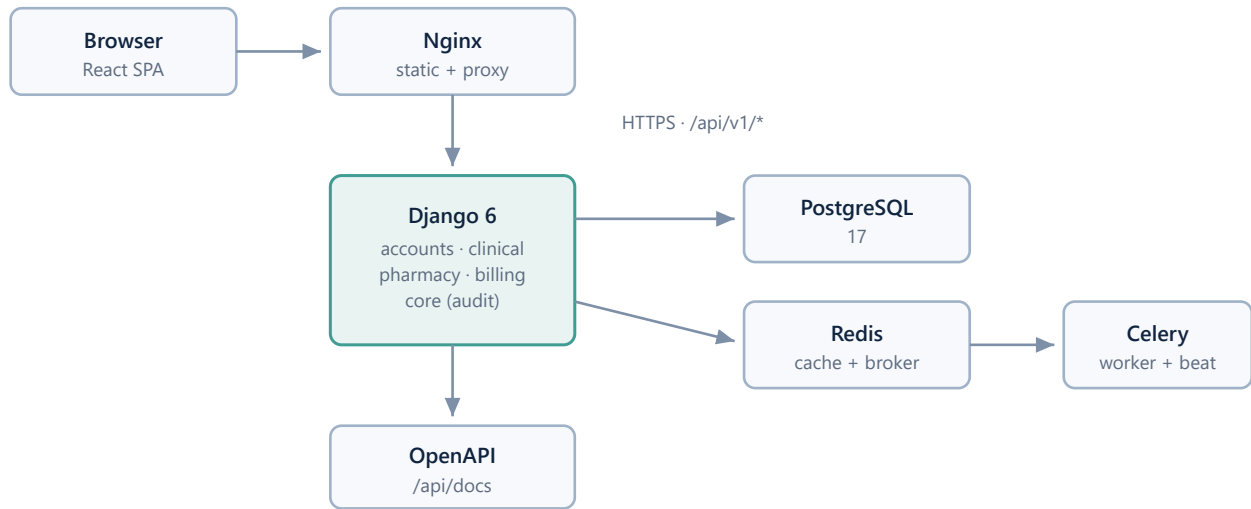
Model		Changes
<code>Patient</code>	extend	Unique medical record number, emergency contact, allergies, chronic conditions. Replace the stored <code>age</code> integer with <code>date_of_birth</code> and derive age — a stored age is wrong the day after you save it.
<code>Doctor</code>	extend	Licence number, consultation fee, qualification, slot duration.
<code>DoctorSchedule</code>	NEW	Weekday, start, end, slot length. Defines when a doctor is bookable and drives slot generation.
<code>Appointment</code>	extend	Reason, duration, cancellation reason, check-in time, and the uniqueness constraint from H-6.

Prescription	extend	<code>prescribed_by</code> , follow-up date, status (draft / issued / dispensed).
Medicine	extend	Generic name, manufacturer, unit price, prescription-required flag.
MedicineStock	NEW	Batch, quantity on hand, reorder level, expiry. Your README calls the current model an inventory, but it has no quantities — it's a catalogue.
Bill	rework	Invoice number, link to appointment, subtotal / tax / discount / total, status, due date. Total derives from line items rather than being typed in.
BillItem , Payment	NEW	Itemised charges, and payments that support partial settlement.
AuditLog	NEW	Actor, action, target, field-level diff, IP, timestamp. Append-only — saving over or deleting a row raises.
Notification	NEW	Recipient, message, read flag. Backs in-app alerts and the email queue.

5.3 Features worth adding

- **Slot-based booking.** `/doctors/{id}/available-slots/?date=` returns genuinely free slots. Booking a taken one returns 409.
- **Role dashboards.** One endpoint, role-appropriate payload — today's schedule for a doctor, the queue and unpaid invoices for reception, revenue and low stock for an admin. This is the single biggest visible upgrade to the app.
- **Real invoicing.** Line items, partial payments, and a printable PDF invoice.
- **Pharmacy dispensing.** Decrement stock atomically, oldest non-expired batch first, with alerts on low stock and near expiry.
- **Notifications.** Booking confirmations and 24-hour reminders, sent through Celery so the request never waits on SMTP.
- **Patient history timeline** and admin reports (revenue by department, doctor utilisation, no-show rate) exportable to CSV.

5.4 Target architecture



Stay a monolith — at this scale it's the right call, and bounded Django apps give you the modularity without the cost of distributed systems. Move to PostgreSQL because you need row locking for slot booking and JSON for the audit diff. Add Redis only once Celery and dashboard caching actually justify it. Most importantly, **keep authorisation in querysets and managers rather than in views**, so a new endpoint is scoped correctly by default instead of by you remembering to do it.

6 Testing

You currently have none, and it's what makes everything in §2 believable.

The single most valuable thing you can write is a **permission matrix test**: loop every combination of role × resource × method × ownership and assert the status code. That's roughly 200 cases from one parametrised test, and it turns §5.1 from a table in a document into something the build enforces.

- `pytest` + `pytest-django` + `factory_boy`, with a fixture per role.
- Constraint tests: double booking, past dates, negative stock, bill totals.
- Serializer tests for nested prescription create *and* update.
- `assertNumQueries` caps on list endpoints so N+1s can't come back.
- Vitest for the auth context and refresh interceptor; one Playwright run through book → consult → prescribe → bill → pay.
- Target ≥ 80% backend coverage overall, and **100% on** `permissions.py`.

7 Build order

Phases 0 to 2 are gating. Don't deploy anywhere reachable until phase 1 is done.

Phase	Time	Done when
0 • Stop the bleeding	2–3 days	C-1, C-4, C-5, C-6, H-1, H-8. No privilege escalation, no secrets in git, project installs from a manifest and imports on Python 3.13.
1 • Authorisation	1–1.5 wk	C-2, C-3, JWT lifecycle, audit log, and the permission matrix test green.
2 • API platform	1 wk	H-2, H-3, N+1 fixes, versioned <code>/api/v1/</code> , OpenAPI published.
3 • Data model	1–1.5 wk	§5.2 migrations, H-6, H-10, soft delete in place.
4 • Frontend	1.5–2 wk	H-4, H-5, TanStack Query, role-aware nav, real loading and error states.
5 • Features	2–3 wk	§5.3 — dashboards, invoicing, pharmacy, notifications, calendar.
6 • Production	1 wk	Docker Compose, CI green on all gates, coverage ≥ 80%, backup and restore rehearsed.

If you only do one week of this

Do phase 0 and the `get_queryset()` scoping from C-2. That's the difference between a project that leaks every patient record to anyone who signs up and one that doesn't — and for a portfolio, being able to explain *why* you added record-level scoping is worth more than any feature in phase 5.

8 Dependencies and verification

Backend

- `django-environ` — config
- `psycopg[binary]` — Postgres
- `drf-spectacular` — OpenAPI
- `celery`, `redis`, `django-redis`
- `gunicorn`, `whitenoise`
- `argon2-cffi` — hashing
- `weasyprint` — invoice PDFs
- `sentry-sdk` — errors
- `pytest`, `pytest-django`, `pytest-cov`, `factory_boy`
- `ruff`, `mypy`, `bandit`, `pip-audit`

Frontend

- `@tanstack/react-query`
- `react-hook-form` + `zod`
- `recharts` — dashboard charts
- `date-fns`, `sonner`
- `vitest` + Testing Library
- `@playwright/test`
- `prettier`

Infrastructure

- PostgreSQL 17 · Redis 7 · Nginx
- Docker Compose · GitHub Actions

Checklist before you call any phase done

- ☐ Tests written and passing; coverage hasn't dropped.
- ☐ Every new endpoint has a row in the permission matrix test.
- ☐ Migrations included and reversible.
- ☐ `ruff`, `mypy` and `eslint` clean.
- ☐ `manage.py check --deploy` still reports zero issues.
- ☐ No secrets, patient data or debug output added.